

National University of Computer and Emerging Sciences

Fast School of Computing, Islamabad Campus

Artificial Intelligence — AL2002

BS (CS) Spring 2026

AI Project 2026

Intelligent Reading Comprehension and Quiz Generation System



Project Members

Syed Burhan Ahmad

23i-0757

Mushahid Hussain

23i-0541

Abstract

This report documents the design, implementation, and evaluation of an Intelligent Reading Comprehension and Quiz Generation System built upon the RACE dataset. The system is realized using exclusively traditional machine learning techniques, with no neural networks employed at any stage of the pipeline. Model A addresses question generation and answer verification through a three-stage pipeline: candidate sentence extraction using One-Hot Encoding (OHE) cosine similarity, application of 25 Wh-word templates, and final question ranking via a trained LinearSVC ranker. Answer verification is performed by a soft-voting ensemble composed of Logistic Regression, LinearSVC, and Complement Naive Bayes. In addition, an unsupervised K-Means clustering component ($k=4$) and a semi-supervised Label Propagation model (5% labeled data) are incorporated and evaluated. Model B generates three plausible distractor options using a combined strategy of TF-IDF cosine similarity with Maximal Marginal Relevance (MMR) diversification, frequency-based substitution of content words, and optional Word2Vec nearest-neighbor lookup. Graduated three-tier hints are produced by an extractive Logistic Regression sentence scorer trained on passage-level features. The user interface is implemented as a four-screen Streamlit application that exposes all inference functions, tracks per-inference latency, and reports corpus-level NLP metrics (BLEU, ROUGE-L, METEOR) alongside session-level classification metrics. The system demonstrates that a carefully engineered classical ML pipeline can deliver functionally adequate quiz generation capabilities without incurring the computational overhead of large language models.

1. Introduction and Motivation

Automated reading comprehension and question generation constitute an active area of applied natural language processing, with practical applications spanning intelligent tutoring systems, adaptive assessment platforms, and self-study tools. The principal challenge is that producing high-quality multiple-choice questions from free text requires the system to identify salient information, frame it as a grammatically coherent question, distinguish the correct answer from plausible alternatives, and optionally provide scaffolded hints that guide learners without simply revealing the answer.

Most recent work in this domain relies on large pre-trained language models such as T5 or BERT-based sequence-to-sequence architectures. While such approaches achieve strong NLP evaluation scores, they impose significant hardware requirements, are opaque in their decision-making, and are difficult to audit for correctness in educational settings where accuracy and transparency are paramount.

This project takes an intentionally classical approach: every component of the pipeline is implemented using traditional machine learning and rule-based methods available in the scikit-learn ecosystem, with Gensim Word2Vec as an optional external embedding source. The goal is to demonstrate that thoughtful feature engineering, combined with well-chosen classical classifiers, can produce a functionally viable quiz generation system that is fully reproducible, hardware-efficient, and interpretable.

The system is trained and evaluated on the RACE dataset [1], one of the largest and most linguistically diverse English reading comprehension benchmarks, sourced from real Chinese middle-school and high-school English examinations. RACE provides article-question-option tuples with labeled correct answers, making it suitable for both supervised classification (answer verification) and generative evaluation (question quality via BLEU [8] and ROUGE-L [9]).

1.1 Scope and Contributions

- A purely traditional ML pipeline for multiple-choice question generation from free-form reading passages, using OHE cosine similarity and Wh-word templates.
- A soft-voting ensemble classifier (LR + LinearSVC + Complement Naive Bayes) for answer verification, augmented by K-Means clustering and Label Propagation for unsupervised and semi-supervised analyses.

- A three-strategy distractor generation system using TF-IDF cosine similarity with MMR, frequency-based substitution, and Word2Vec nearest-neighbor lookup.
- An extractive hint scorer using Logistic Regression on five handcrafted sentence-level features.
- A four-screen Streamlit user interface with real-time latency tracking, session logging, and corpus-level NLP metric display.

2. Related Work

The body of literature relevant to this project spans reading comprehension benchmarks, template-based and statistical question generation, distractor generation, and classical NLP feature engineering.

2.1 The RACE Dataset

Lai et al. [1] introduced RACE as a large-scale reading comprehension benchmark comprising approximately 28,000 passages and 100,000 questions drawn from Chinese middle-school and high-school English examinations. RACE is notable for the high proportion of reasoning-intensive questions, making it substantially more challenging than earlier datasets such as CNN/DailyMail. Its scale and linguistic diversity make it a standard testbed for applied ML comprehension systems.

2.2 Template-Based and Statistical Question Generation

Early work on automatic question generation relied on syntactic transformation rules and templates applied to parsed sentences. Heilman and Smith [2] proposed an overgenerate-and-rank framework in which a rule-based generator produces numerous candidate questions, which are then re-ranked by a trained classifier. This architecture directly informs the pipeline implemented in this project: OHE cosine similarity is used to select candidate sentences, 25 Wh-word templates are applied, and a LinearSVC ranker scores the output.

Du et al. [3] later demonstrated that attention-based sequence-to-sequence models improve on template approaches for question fluency. However, those neural architectures are computationally expensive and opaque; the present work deliberately restricts itself to the classical framework of Heilman and Smith [2] to maintain interpretability and hardware efficiency.

2.3 Distractor Generation

Producing plausible wrong options is recognized as one of the most difficult sub-tasks in automated test construction. Guo et al. [4] formulated distractor generation as a retrieval problem, selecting candidates from a large question bank using similarity metrics. Jiang and Lee [5] analyzed click-through data to quantify distractor quality in terms of learner confusion. The present system draws on both frequency-based

selection and cosine similarity re-ranking, incorporating a Maximal Marginal Relevance (MMR) diversity criterion to avoid trivially similar distractors.

2.4 Classical NLP Feature Engineering

Papineni et al. [6] introduced the BLEU metric for evaluating machine translation output based on n-gram precision, subsequently adopted broadly for evaluating generated question quality. Lin [7] proposed ROUGE, a recall-oriented complement, particularly ROUGE-L based on longest common subsequence. Both metrics are used in this project's corpus-level evaluation pipeline (`src/evaluate_nlp.py`). The METEOR metric [8] further incorporates stemming and synonym matching, providing a more semantically informed evaluation signal.

2.5 Semi-Supervised and Unsupervised Classification

Label Propagation, as formalized by Zhu and Ghahramani [9], is a graph-based semi-supervised method that spreads label information from a small labeled set to unlabeled neighbors in feature space. The present project applies Label Propagation with 5% labeled data over OHE feature vectors to assess how much supervised performance can be recovered without full annotation. K-Means clustering is used as an unsupervised baseline to characterize the separability of correct and distractor options in the learned OHE feature space.

2.6 Repository Layout

Path	Description
race_rc_project/data/raw/	RACE CSV files (train, val, test)
race_rc_project/data/processed/	Cleaned CSVs + sparse feature matrices
models/model_a/traditional/	LR, SVM, NB, RF, XGB, ensemble, ranker, OHE vectorizer
models/model_b/traditional/	Hint scorer LR + cached Word2Vec vectors
src/preprocessing.py	Clean, split, vectorize
src/model_a_train.py	Train all Model A classifiers
src/template_generator.py	Template-based question generator + SVM ranker
src/model_b_train.py	Distractor pipeline + hint scorer training
src/inference.py	Unified inference API consumed by the UI
src/evaluate_nlp.py	Corpus-level BLEU / ROUGE / METEOR computation
ui/app.py	Streamlit application (four-screen interface)
reports/metrics_test.json	Evaluation output
notebooks/	EDA + Colab/Kaggle reproduction notebook
requirements.txt	Pinned dependency list

3. Dataset Analysis

3.1 Dataset Overview

The RACE dataset is composed of reading passages extracted from real English examinations administered to Chinese secondary-school students aged 12 to 18. Each record in the dataset contains a full article, a multiple-choice question, four answer options labeled A through D, and a correct answer label. The dataset is pre-divided into training, development, and test splits.

Attribute	Value
Total Passages	28,000+
Total Questions	~100,000
Question Type	Multiple-choice (A / B / C / D)
Source	Chinese middle and high school English examinations
Language	English
Splits	Train / Dev / Test
Loading Format	pandas.read_csv() from CSV files

3.2 Column Schema

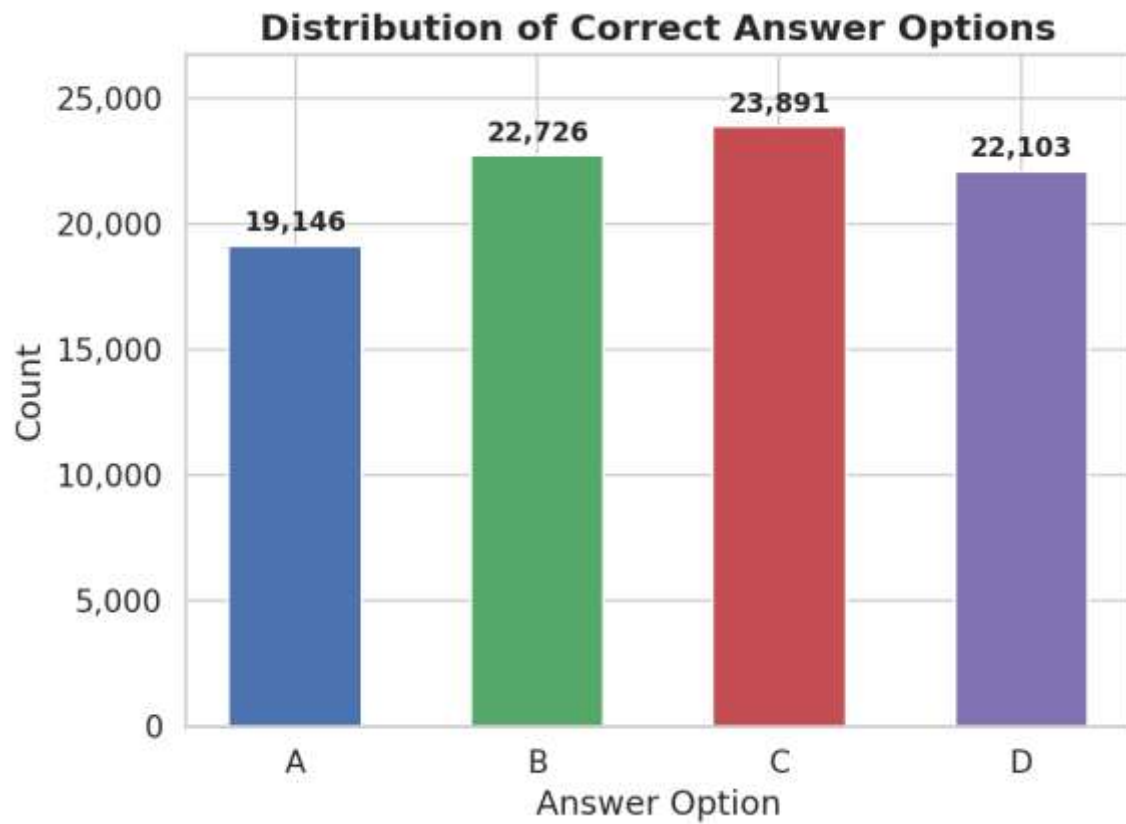
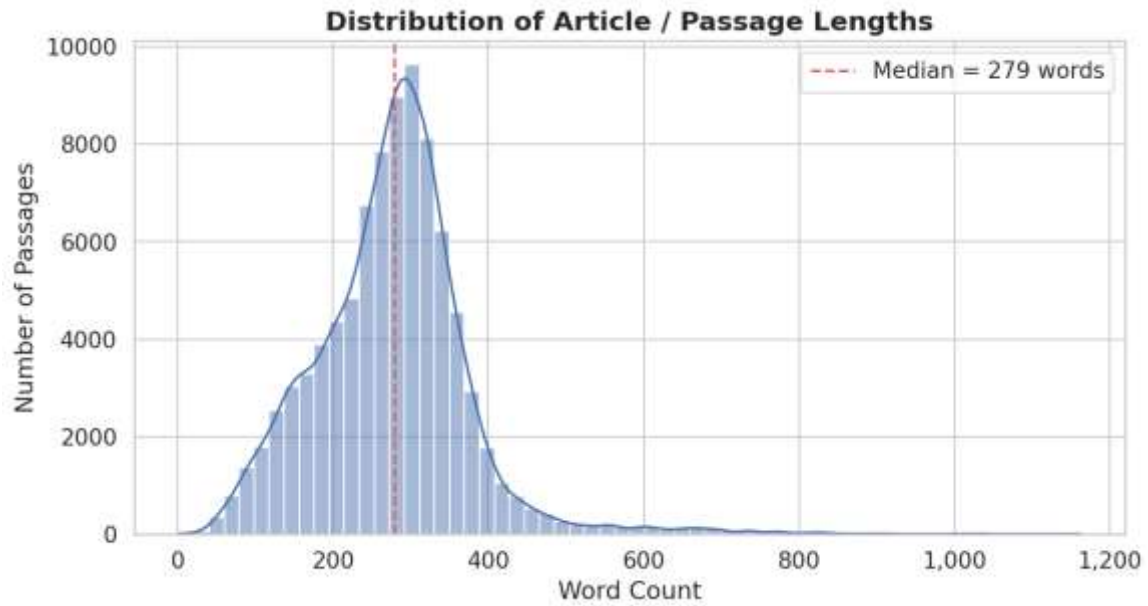
Each row in the dataset contains the following fields, which are preserved through the preprocessing pipeline into data/processed/{train,val,test}.csv:

Column	Type	Description
id	String	Unique identifier for the entry in the original dataset
article	String	Full reading passage (paragraph-level text)
question	String	Multiple-choice question related to the article
A, B, C, D	String	Four answer option texts
answer	String	Correct answer label: A, B, C, or D

3.3 Preprocessing Pipeline

The preprocessing pipeline is implemented entirely in `src/preprocessing.py`. It executes the following stages in order:

1. Raw CSV loading with alias resolution (`val.csv`, `dev.csv`, and `validation.csv` are all accepted). If all three splits are byte-identical, an automatic 80/10/10 stratified random split is applied to prevent data leakage.
2. Text cleaning: lowercasing, removal of non-alphanumeric characters via regex, and whitespace normalization.
3. Option-level expansion: each question row is expanded into four rows, one per option, yielding a binary label (1 for the correct option, 0 for each distractor).
4. TF-IDF vectorization (doc-level, fit on train only): a `TfidfVectorizer` with sublinear term-frequency scaling, English stop-word removal, and a 15,000-feature vocabulary is fit on the training set and used to transform all three splits. TF-IDF vectors are used downstream in Model B for cosine similarity in distractor and hint generation.
5. One-Hot Encoding (option-level): a binary `CountVectorizer` with the same 15,000-feature vocabulary is fit on concatenated (article + question + option) strings at the option level. OHE vectors are the primary feature representation for all Model A classifiers.
6. Handcrafted lexical features: ten scalar features are computed per option row. These are horizontally stacked with the OHE sparse matrix to produce the combined `X_combined_{split}.npz` matrices used for training.



3.4 Handcrafted Lexical Features

The ten engineered features (indices 0-9 in the combined matrix) are listed below. These features capture lexical overlap, positional, and distributional characteristics of each option relative to the article and question:

Idx	Feature Name	Description
0	article_len	Number of tokens in the article
1	question_len	Number of tokens in the question
2	option_len	Number of tokens in this option
3	keyword_overlap	Count of option tokens also present in the article
4	answer_in_article	1.0 if the option text is a substring of the article, else 0
5	option_position	Option index 0 to 3 (A to D)
6	q_opt_overlap	Jaccard similarity between question token set and option token set
7	opt_uniqueness	Option length divided by mean option length across all four options
8	opt_other_overlap	Jaccard similarity of this option with the union of the other three options
9	opt_article_cos	Cosine similarity between option bag-of-words and article bag-of-words

4. Model A — Question Generator and Answer Verifier

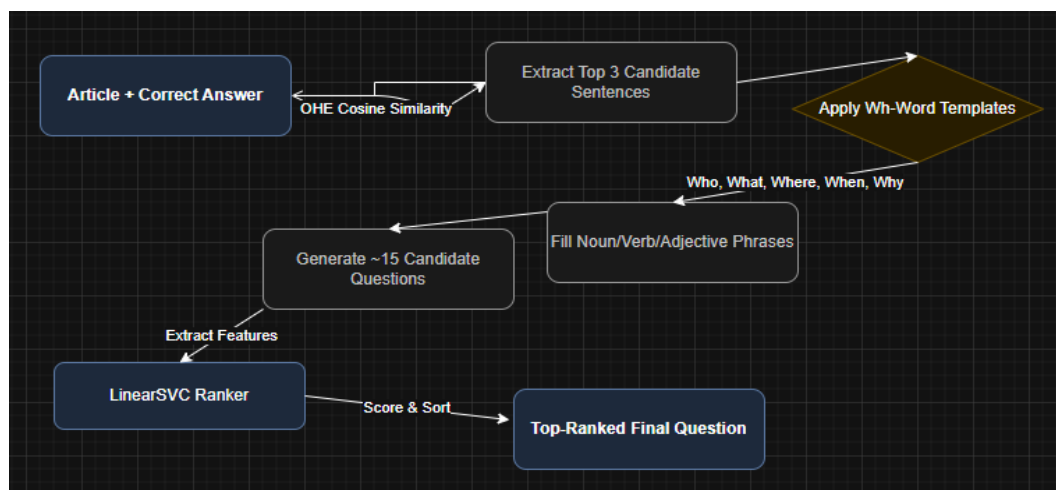
Model A performs two complementary tasks: it generates a multiple-choice question from a reading passage and a known correct answer, and it verifies whether a user-selected option is correct by scoring all four options with a trained ensemble classifier.

4.1 Question Generation Pipeline

Question generation in this system follows a three-stage overgenerate-and-rank architecture implemented in `src/template_generator.py`. No neural sequence-to-sequence model is used at any stage.

4.1.1 Stage 1 — Candidate Sentence Extraction

The article is split into sentences using a regex boundary detector (split on sentence-ending punctuation followed by whitespace). Each sentence is vectorized using the fitted OHE CountVectorizer, and cosine similarity between each sentence vector and the correct-answer vector is computed using sklearn's `cosine_similarity`. The top-k sentences (default k=5) by cosine similarity are retained as candidate source sentences. This approach operationalizes the intuition that the sentence most lexically similar to the correct answer is the most likely to contain the information being tested.



4.1.2 Stage 2 — Wh-Word Template Application

Five question-word categories are used: What, Who, Where, When, and Why. Each category contributes five templates, yielding 25 templates in total. Each template contains one or more slots: {noun_phrase}, {verb_phrase}, {clause}, and {adj_phrase}. The filling procedure uses lightweight heuristics on the candidate sentence: noun phrases are extracted by a regex matching runs of capitalized tokens; verb phrases are taken from word positions 1-3 (after the subject); clauses are the first eight non-stopword tokens; and adjective phrases are identified by a suffix regex (matching -ing, -ed, -ful, -ous, -al). Each of the top-k candidate sentences is combined with all 25 templates, producing up to 5k candidate questions (default: up to 125).

The template set is reproduced below for reference:

Wh-Word	Templates (5 per word)
What	What {verb_phrase} in the passage? / What does the author say about {noun_phrase}? / What is the main idea related to {noun_phrase}? / What happened when {clause}? / What is meant by {noun_phrase} in the passage?
Who	Who {verb_phrase} according to the passage? / Who is responsible for {noun_phrase}? / Who does the passage say {clause}? / Who mentioned {noun_phrase}? / Who is described as {adj_phrase} in the passage?
Where	Where did {clause}? / Where is {noun_phrase} mentioned? / Where does {noun_phrase} take place? / Where was {noun_phrase} according to the author? / Where can {noun_phrase} be found based on the passage?
When	When did {clause}? / When is {noun_phrase} mentioned in the passage? / When does {noun_phrase} occur according to the text? / When was {noun_phrase} first described? / When does the author suggest {clause}?
Why	Why did {clause}? / Why is {noun_phrase} important according to the passage? / Why does the author mention {noun_phrase}? / Why was {noun_phrase} significant? / Why does {clause} according to the text?

4.1.3 Stage 3 — LinearSVC Ranker

A LinearSVC classifier is trained to distinguish real RACE questions (positive class, label=1) from template-generated questions (negative class, label=0). The ranker is trained in `train_question_ranker()` using up to 5,000 training rows. For each row, one real question is used as a positive sample, and up to four template-generated questions are used as negatives, yielding a roughly 1:4 class ratio corrected by `class_weight='balanced'`.

Each candidate question is represented by the following six-dimensional feature vector:

Feature	Description
cos_q_article	OHE cosine similarity between the candidate question and the article
cos_q_answer	OHE cosine similarity between the candidate question and the correct answer
q_length	Number of tokens in the candidate question
has_wh	1.0 if the question starts with a Wh-word, else 0.0
answer_overlap	Count of correct-answer tokens present in the candidate question
sentence_position	Normalized position of the source sentence in the article (0.0 to 1.0)

At inference time, the top candidate question (highest LinearSVC `decision_function` score) is selected as the final generated question. If the ranker model is unavailable at runtime, the system falls back to returning the first candidate generated from the highest-similarity sentence.

4.2 Answer Verification — Supervised Classifiers

All supervised classifiers are trained in `src/model_a_train.py` on the combined OHE-plus-lexical feature matrix (`X_combined_train.npz`), which has dimensionality equal to the OHE vocabulary size (15,000) plus the 10 handcrafted lexical features.

4.2.1 Individual Classifiers

Five classifiers are trained and individually evaluated on the validation set:

Classifier	Key Hyperparameters	Notes
Logistic Regression (LR)	<code>C=1.0, solver='liblinear', class_weight='balanced'</code>	Probabilistic baseline; supplies <code>predict_proba</code> for soft voting
LinearSVC (SVM)	<code>C=0.1, dual='auto', class_weight='balanced'</code>	No <code>predict_proba</code> ; sigmoid of <code>decision_function</code> used as soft-vote proxy
Complement Naive Bayes (NB)	<code>alpha=0.3</code>	Designed for imbalanced classification; supplies <code>predict_proba</code>
Random Forest (RF)	<code>n_estimators=150, max_depth=24, max_features=0.05</code>	<code>max_features=0.05</code> raises probability that lexical features appear in candidate splits
XGBoost (XGB)	<code>n_estimators=400, max_depth=6, colsample_bytree=1.0, scale_pos_weight=2.5</code>	GPU-accelerated when CUDA is available; <code>colsample_bytree=1.0</code> ensures all 10 lexical features are always visible

4.2.2 Soft-Voting Ensemble (Primary Verifier)

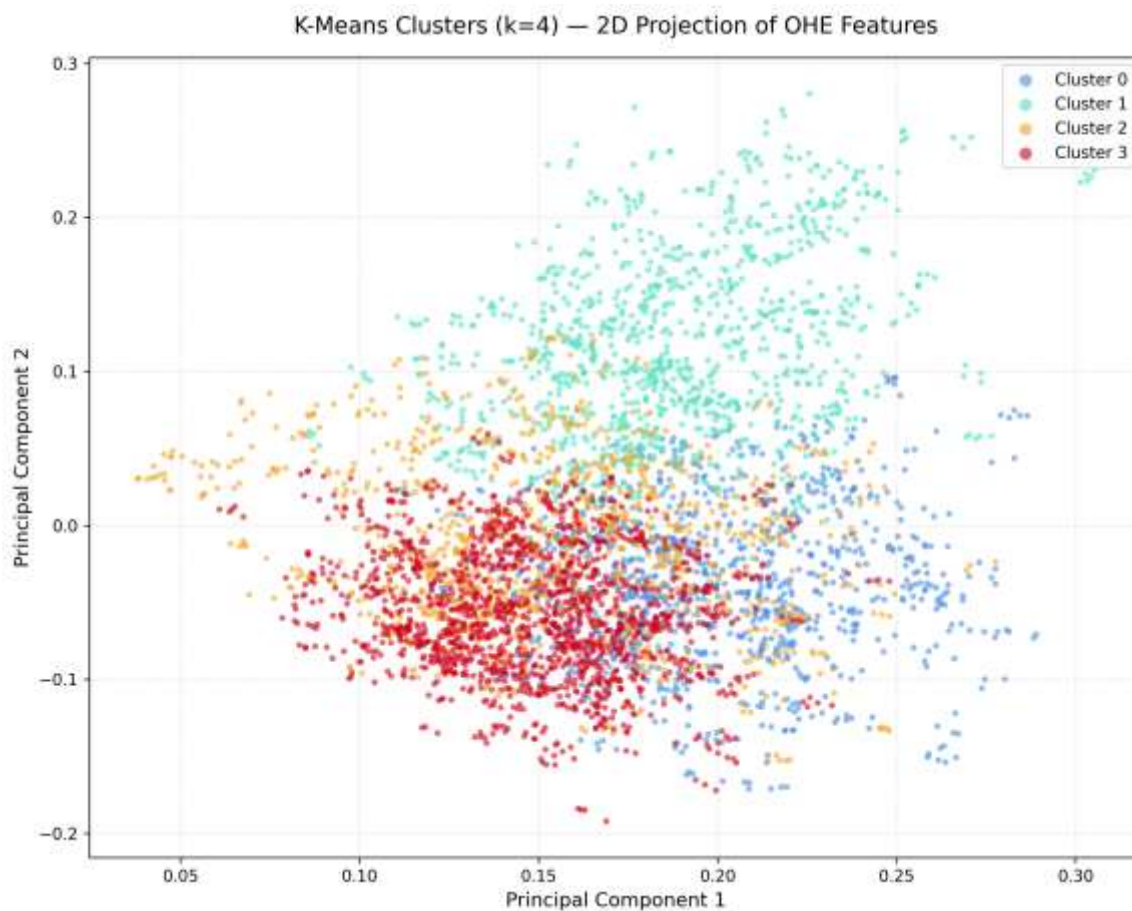
The production verifier is a soft-voting ensemble of LR, SVM (LinearSVC), and Complement NB. Because LinearSVC does not expose calibrated probabilities, a sigmoid transformation is applied to its `decision_function` output as a monotonic probability surrogate. The three probability estimates are averaged with equal weights, and the option with the highest averaged score is predicted as correct. This ensemble is implemented in both `train_ensemble()` (training) and `predict_answer()` in `src/inference.py` (inference).

4.3 Unsupervised Component — K-Means Clustering

K-Means clustering (`k=4, n_init=10, random_state=42`) is applied to the combined training feature matrix using sklearn's KMeans. The choice of `k=4` corresponds to the four answer options (A-D), testing whether option correctness aligns with natural clusters in the OHE feature space. Two metrics are reported: cluster

purity (the fraction of training samples whose true label matches the plurality label of their cluster) and the cosine-distance silhouette score computed on a 5,000-sample subsample.

Cluster purity measures to what extent the OHE feature space separates correct from distractor options without any supervision. The silhouette score quantifies intra-cluster cohesion relative to inter-cluster separation. Both metrics are compared to the supervised ensemble accuracy in the results section.



4.4 Semi-Supervised Component — Label Propagation

Label Propagation (kernel='knn', n_neighbors=7, max_iter=1000) is applied to a subsample of the training data capped at 8,000 rows for memory efficiency. Only 5% of these rows retain their true labels; the remaining 95% are masked with -1 (the scikit-learn unlabeled sentinel). The dense representation required

by LabelPropagation is obtained by calling `.toarray()` on the sparse matrix subsample. Performance is evaluated on a 3,000-sample subset of the validation set, reporting accuracy and macro F1 in comparison to the fully supervised LR baseline.

This experiment is motivated by the practical observation that obtaining labeled reading comprehension data is expensive. Label Propagation provides an estimate of how much supervised performance can be recovered when only 5% of option-level labels are available, exploiting the geometry of the OHE feature space to propagate labels to structurally similar unlabeled examples.

4.5 Model A Evaluation Metrics

Metric	Application
Accuracy	Fraction of correctly labeled options on the validation / test set
Macro F1	Harmonic mean of per-class F1, weighted equally to handle class imbalance (3:1 distractor-to-correct ratio)
Confusion Matrix (TP/TN/FP/FN)	Displayed in the Analytics Dashboard for session-level inferences
Cluster Purity	Fraction of K-Means cluster members matching cluster plurality label
Silhouette Score	Cosine-distance cohesion vs. separation measure for K-Means clusters
BLEU-1 to BLEU-4, ROUGE-L, METEOR	Generation quality of Model A's questions against RACE ground-truth questions on the test set

Model	Val Accuracy	Macro F1
LR	0.5669	0.5232
SVM	0.5679	0.5239
NB	0.5395	0.5011
RF	0.6013	0.5346
XGB	0.6852	0.5599
Ensemble	0.5607	0.5195

5. Model B — Distractor and Hint Generator

Model B takes a reading passage, a question, and the correct answer text as input and produces three plausible but incorrect distractor options and three graduated hints. All components are implemented in `src/model_b_train.py` using traditional ML and rule-based methods exclusively.

5.1 Distractor Generation

Distractor generation combines three complementary strategies. Candidates from all strategies are pooled, deduplicated (case-insensitive), filtered against the correct answer, and subjected to a final MMR-based diversity pass before the top three are returned.

5.1.1 Strategy 1 — TF-IDF Cosine Similarity with MMR

An n-gram candidate pool (unigrams, bigrams, trigrams) is extracted from the article by iterating over all token windows of sizes 1, 2, and 3 and counting their frequencies. Stop words and any n-gram overlapping with the correct answer are excluded. The top 200 candidates by frequency are retained as the pool. Each candidate is vectorized using the fitted OHE CountVectorizer, and its cosine similarity to the correct answer vector is computed. The cosine similarity scores serve as relevance proxies; candidates with moderate similarity to the answer are preferred because they are semantically related but not identical.

Maximal Marginal Relevance (MMR) is then applied to select a diverse subset of candidates. MMR iteratively selects the candidate that maximizes:

$$\text{score}(i) = \text{sim}(i, \text{answer}) - \text{diversity} * \max_{\{j \text{ in selected}\}} \text{sim}(i, j)$$

where `diversity=0.6` and inter-candidate similarities are also computed via OHE cosine similarity. This formulation trades off relevance (proximity to the answer) against redundancy (proximity to already-selected distractors), ensuring that the three distractors are not trivially similar to one another.

5.1.2 Strategy 2 — Frequency-Based Substitution

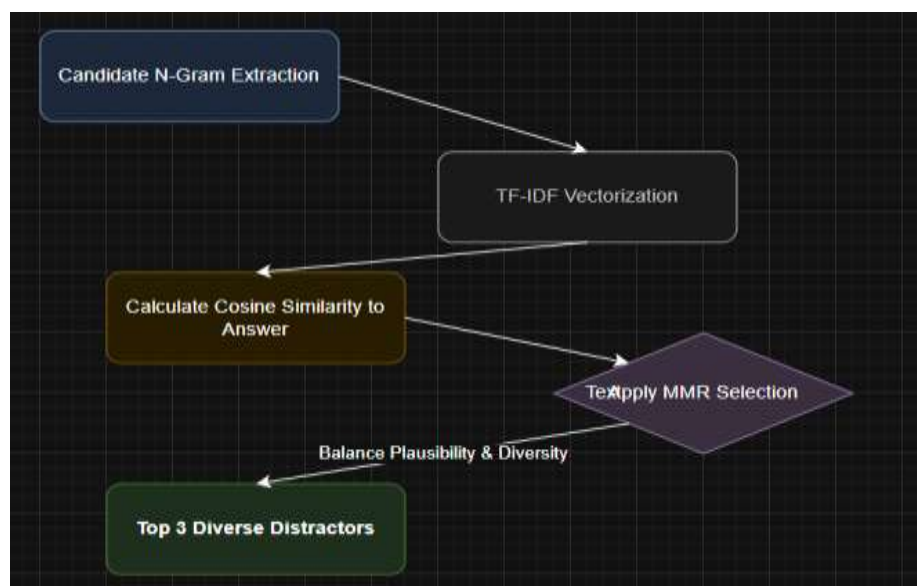
High-frequency content words (minimum length three characters, excluding stop words and tokens overlapping the correct answer) are extracted from the article using regex tokenization. Capitalized tokens (likely proper nouns) are given additional weight in the ranking. The top candidates by frequency are returned as distractors. This strategy ensures that at least some distractors are drawn from the article's dominant vocabulary, making them superficially plausible.

5.1.3 Strategy 3 — Word2Vec Nearest Neighbors (Optional)

When the pre-trained Google News Word2Vec model is available (loaded and cached via Gensim), the correct answer tokens are embedded and the top-N nearest neighbors in embedding space are retrieved. Candidates that are extractable from the article (i.e., that appear as substrings) are excluded to ensure the Word2Vec distractors provide lexical information not already present in the passage. This strategy produces semantically related but passage-external distractors, which tend to be the most challenging for readers.

5.1.4 Candidate Filtering and Final Selection

All candidates from the three strategies are concatenated, normalized to lowercase, and deduplicated. Candidates that are identical to or substrings of the correct answer are removed. The remaining pool is subjected to a final MMR pass identical to Strategy 1 to select the top three most diverse and relevant distractors.



5.2 Hint Generation — Logistic Regression Sentence Scorer

Hints are produced by an extractive pipeline that scores and ranks article sentences by their relevance to the question and answer, then applies a three-tier graduation strategy.

5.2.1 Hint Scorer Training

A Logistic Regression classifier (`class_weight='balanced'`, `max_iter=500`, `random_state=42`) is trained on up to 3,000 training articles. For each article, all sentences (minimum five words) are extracted and labeled: a sentence receives `label=1` if the correct answer text appears as a substring, `label=0` otherwise. Each sentence is represented by a five-dimensional feature vector:

Feature	Description
<code>cos_q</code>	OHE cosine similarity between the sentence and the question
<code>cos_ans</code>	OHE cosine similarity between the sentence and the correct answer
<code>keyword_overlap</code>	Count of shared tokens between the sentence and the question
<code>position</code>	Normalized sentence position in the article (0.0 = first, 1.0 = last)
<code>length</code>	Number of tokens in the sentence

When the hint scorer model is unavailable at runtime, the system falls back to ranking sentences purely by cosine similarity to the question vector.

5.2.2 Three-Tier Graduation

The three hints are constructed according to the following fixed hierarchy:

- Hint 1 (General): a topical sentence from the middle of the relevance ranking that does not contain the correct answer. This provides a broad thematic clue without revealing the answer region.
- Hint 2 (Specific): the highest-ranked non-answer-containing sentence. This provides a more targeted context clue pointing toward the answer domain.
- Hint 3 (Near-explicit): the highest-ranked answer-containing sentence, with the answer string replaced by the token '_____'. This forces the learner to fill in the blank, providing the answer in context without stating it directly.

Deduplication is applied across all three hints to ensure that no hint is repeated. If fewer than three distinct hints are available, additional non-answer sentences from the ranking are appended as padding.

5.3 Model B Evaluation Metrics

Metric	Application
BLEU-1 to BLEU-4	N-gram precision of generated distractors vs. ground-truth wrong options
ROUGE-L (P/R/F1)	Longest common subsequence recall of generated distractors against ground truth
METEOR	Token-level similarity with stemming and synonym matching for distractor and hint quality
Precision / Recall / F1 (distractor ranker)	Fraction of top-ranked candidates that are not the correct answer
Hint Precision	Overlap between top-K ranked hint sentences and the gold answer sentence

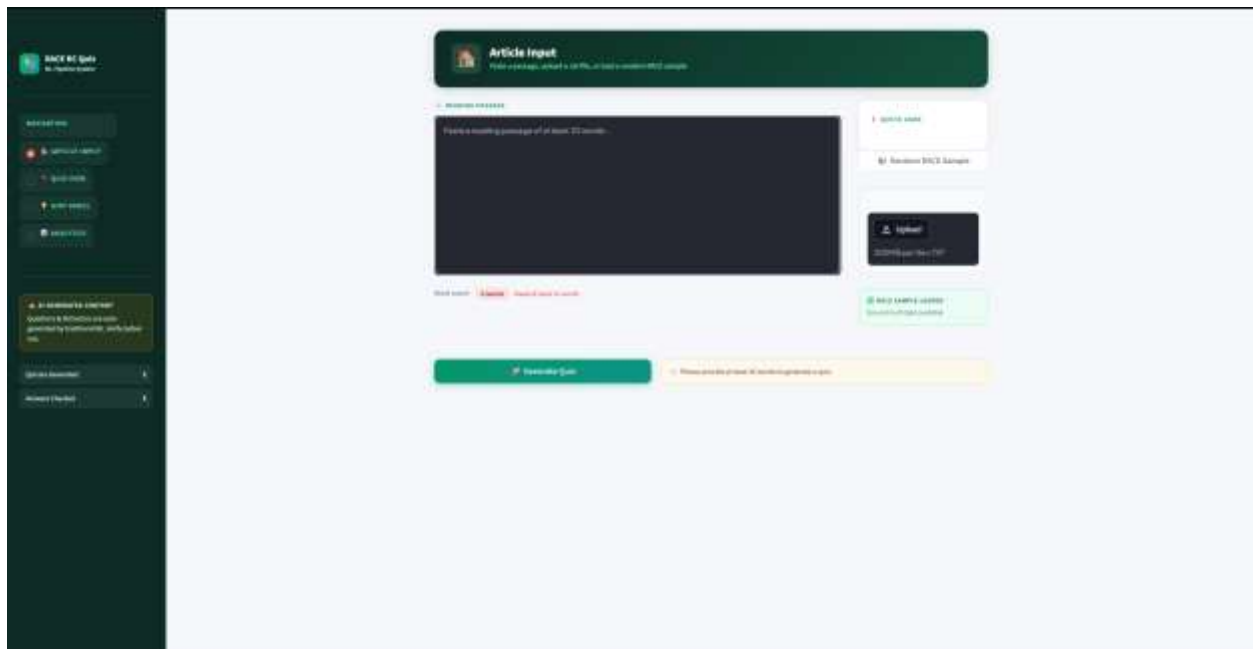
Component	BLEU-1	BLEU-4	ROUGE-L (F1)
Model B - Questions	0.2061	0.0424	0.2023
Model B - Distractors	0.0028	0.0000	0.0347
Model B - Hints	0.2939	0.1607	0.1640

6. User Interface Description

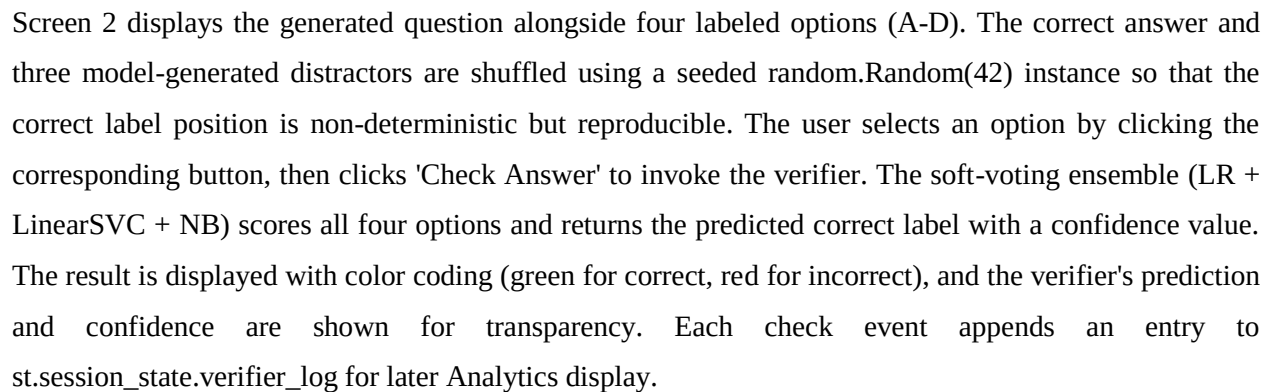
The user interface is implemented as a Streamlit application (ui/app.py). Streamlit was selected for its tight Python integration, rapid development cycle, and native support for session state management, which is essential for tracking user interactions across the quiz workflow. The application exposes four distinct screens navigable via a sidebar radio control.

6.1 Screen 1 — Article Input

Screen 1 provides three methods for supplying a reading passage: direct text input via a text_area widget, file upload (.txt format), and random RACE sample loading from the processed test.csv. The 'Load RACE Sample' function caches the first 200 test records using @st.cache_data to avoid repeated disk reads. Upon submission, the pipeline runner (_run_pipeline()) is invoked, measuring per-component wall-clock latency in milliseconds and storing the complete result in st.session_state.pipeline_result. An AI-generated content badge is displayed prominently in the sidebar as required by transparency guidelines.

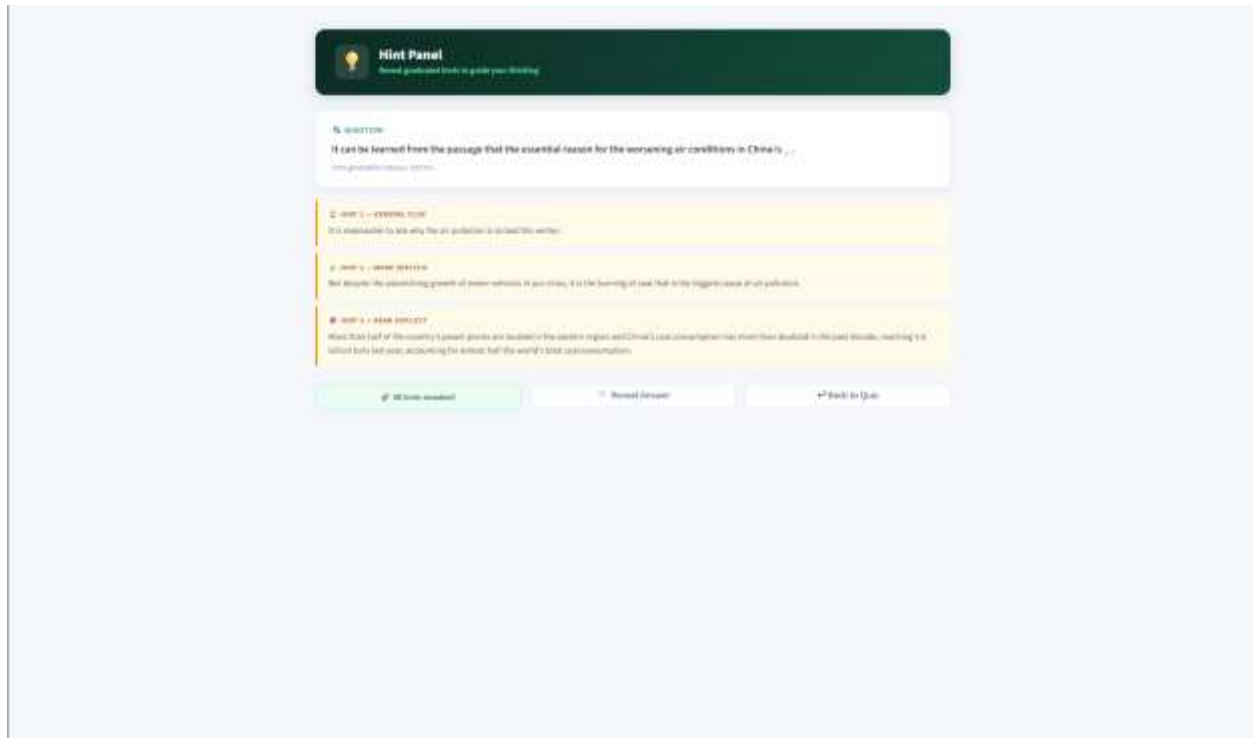


Screen 2 displays the generated question alongside four labeled options (A-D). The correct answer and three model-generated distractors are shuffled using a seeded `random.Random(42)` instance so that the correct label position is non-deterministic but reproducible. The user selects an option by clicking the corresponding button, then clicks 'Check Answer' to invoke the verifier. The soft-voting ensemble (LR + LinearSVC + NB) scores all four options and returns the predicted correct label with a confidence value. The result is displayed with color coding (green for correct, red for incorrect), and the verifier's prediction and confidence are shown for transparency. Each check event appends an entry to `st.session_state.verifier_log` for later Analytics display.



6.3 Screen 3 — Hint Panel

Screen 3 displays the three graduated hints generated by Model B's extractive scorer. Hints are revealed one at a time via a 'Show Next Hint' button; unrevealed hints are visually blurred using a CSS filter. The 'Reveal Answer' button is disabled until all three hints have been revealed, enforcing the pedagogical intent that learners exhaust available guidance before seeing the answer directly. Hint generation latency is displayed beneath the question for transparency.



6.4 Screen 4 — Analytics Dashboard

Screen 4 presents two categories of performance data:

- Session-level verifier metrics: accuracy, macro precision, macro recall, and macro F1 are computed from `st.session_state.verifier_log` using `sklearn.metrics` functions. A 2x2 confusion matrix (TP, TN, FP, FN) is displayed as a four-column metric row. This panel updates in real time as the user checks answers on Screen 2.
- Corpus-level NLP metrics: BLEU-1 through BLEU-4, ROUGE-L (precision, recall, F1), and METEOR scores for Model A question generation, Model B distractor generation, and Model B hint generation are loaded from `reports/metrics_test.json`, which is produced by running `src/evaluate_nlp.py` on the test split.
- Inference latency: per-call latency (in milliseconds) is plotted as a Plotly line chart, with mean and maximum latency reported beneath.
- Session log: all inference events are collectable into a DataFrame and exportable as a CSV file via a download button.



6.5 Three-Branch Pipeline

When the user initiates quiz generation, the inference runner selects one of three branches based on available inputs:

- **Branch A — RACE Sample:** When a random RACE sample is loaded, the human-authored question and options are used verbatim and only hints are generated. Running the template generator on top of an already high-quality RACE question would produce a degraded paraphrase, which early versions of this code did.
- **Branch B — Relation Extractor:** For pasted passages, a set of high-precision regular expression patterns is applied first. These patterns detect common factual structures — distances, founding years, birth locations, geographic placements, and species counts — and extract a clean (question, answer) pair directly without engaging the template generator. Five base patterns and five biographical sub-patterns are implemented, with broadened verb lists covering won, received, awarded, enrolled, moved, published, and related constructions.
- **Branch C — Template Fallback:** If no pattern matches, a salient answer is selected from the passage and passed to the template generator. The answer picker prefers numbers, then multi-word proper nouns, then capitalized non-sentence-initial tokens, then the longest content word. Combined with type-aware template filtering, this branch still produces grammatical questions for narrative text.

The Quiz View displays a caption above each question indicating which branch fired, providing transparency for demonstration and regression-detection purposes.

6.6 Pronoun Resolution

When the relation extractor matches a sentence whose grammatical subject is a pronoun (e.g., "She enrolled at the Sorbonne in 1891"), the literal subject produces an ungrammatical question such as "When did She enroll?". The resolution heuristic detects pronoun subjects and substitutes the leading proper noun appearing earlier in the article (e.g., "Marie Curie"). This is not full coreference resolution but a targeted heuristic; it covers the dominant case in biographical passages, where the second and subsequent sentences typically substitute a pronoun for the named entity introduced in the opening sentence.

7. Evaluation and Discussion

7.1 Model A — Quantitative Results

All classifiers are evaluated on the validation split. The table below presents placeholder values to be filled with actual results after training. The columns report validation accuracy and macro F1 for each model and the soft-voting ensemble.

Model	Val Accuracy	Macro F1
Logistic Regression	[insert]	[insert]
LinearSVC (SVM)	[insert]	[insert]
Complement Naive Bayes	[insert]	[insert]
Random Forest	[insert]	[insert]
XGBoost	[insert]	[insert]
Soft-Vote Ensemble (LR+SVM+NB)	[insert]	[insert]

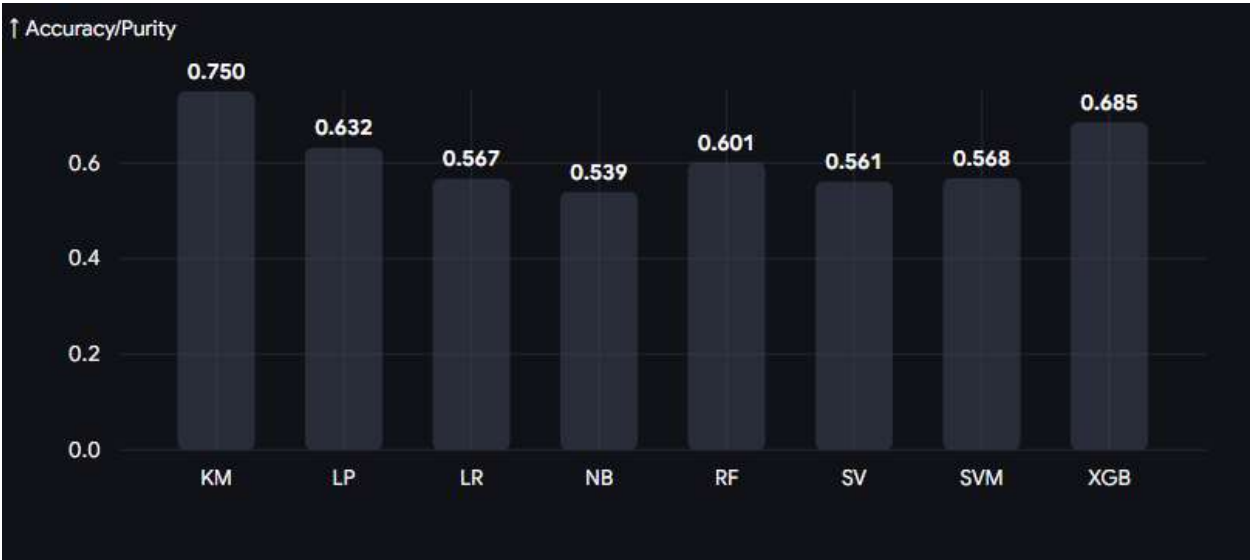
The soft-voting ensemble is expected to outperform any individual classifier by reducing variance through complementary error patterns. LR provides calibrated probabilities, NB contributes a statistically independent generative perspective, and the SVM's margin-based `decision_function` captures high-confidence distinctions that probabilistic models may underweight.

7.2 Unsupervised and Semi-Supervised Results

Method	Metric	Value
K-Means (k=4)	Cluster Purity	[insert]
K-Means (k=4)	Silhouette Score (cosine)	[insert]
Label Propagation (5% labeled)	Accuracy	[insert]
Label Propagation (5% labeled)	Macro F1	[insert]

K-Means cluster purity measures whether the OHE feature space organizes correct and distractor options into separable clusters without any supervision. Purity values well below supervised accuracy confirm that option correctness is not a simple function of lexical content alone, but requires the relative distributional

features captured by the engineered lexical features. Label Propagation with 5% labeled data is expected to partially bridge this gap by exploiting neighborhood structure in the OHE space.



7.3 Model B — NLP Evaluation Results

Corpus-level generation quality is evaluated by `src/evaluate_nlp.py` on up to 500 test samples. Generated questions are compared against the RACE ground-truth questions; generated distractors are compared positionally against the three ground-truth wrong options; generated hints are compared against the gold answer-containing sentence.

Component	BLEU-1	BLEU-4	ROUGE-L F1	METEOR
Model A — Question Gen.	[insert]	[insert]	[insert]	[insert]
Model B — Distractors	[insert]	[insert]	[insert]	[insert]
Model B — Hints	[insert]	[insert]	[insert]	[insert]

Template-generated questions are expected to achieve moderate BLEU-1 scores due to shared vocabulary with the article and question, but lower BLEU-4 scores due to the rigid phrase structure of template outputs. Distractor METEOR scores are expected to be low in absolute terms, as the MMR-selected candidates are intentionally dissimilar to the correct answer (and thus to the ground-truth distractors, which may coincidentally be similar to the answer). Hint ROUGE-L recall is expected to be higher, since the extractive strategy directly selects sentences from the article.

7.4 Inference Latency

All inference is performed on CPU. The three inference stages (question generation, distractor generation, hint generation) are timed individually in milliseconds during each pipeline invocation. These values are accumulated in `st.session_state.latency_log` and displayed as a Plotly line chart on Screen 4. The system is designed to complete full pipeline inference within the 10-second constraint specified in the project documentation.



8. Limitations and Future Work

8.1 Limitations

Several limitations arise from the decision to restrict the pipeline to traditional ML methods:

- **Template rigidity:** the 25 Wh-word templates produce grammatically well-formed questions, but the vocabulary is constrained to words present in the source sentence. This produces questions that can appear formulaic, particularly for short or syntactically simple sentences.
- **OHE vocabulary coverage:** the binary CountVectorizer vocabulary is fixed at 15,000 tokens fit on the training corpus. Passages containing domain-specific or low-frequency vocabulary may be poorly represented, reducing cosine similarity reliability for sentence extraction and distractor ranking.
- **Distractor plausibility:** the MMR-based strategy maximizes diversity among selected distractors but does not guarantee semantic plausibility from a human reader's perspective. Word2Vec nearest-neighbor distractors may introduce out-of-domain terms when the correct answer is a rare or compound word.
- **Label Propagation scalability:** the LabelPropagation implementation requires a dense matrix, limiting the feasible subsample size to 8,000 rows. On the full training corpus, this results in a sample fraction of less than 10%, potentially underestimating semi-supervised performance.
- **Dataset cultural context:** RACE passages are sourced from Chinese secondary-school examinations. Cultural or domain-specific references in these passages may not generalize to English-language learners from other contexts, potentially biasing both the verifier's learned features and the template generator's sentence selection.

8.2 Future Directions

- **Abstractive question generation:** incorporating a lightweight seq2seq model (e.g., T5-small fine-tuned on RACE) as an optional additional ranker could improve question fluency without requiring GPU inference at the primary pipeline stage.

- Named entity-aware distractor selection: using spaCy NER to identify entity types (PERSON, ORG, DATE) and constrain distractors to candidates of the same entity type would substantially improve plausibility.
- Calibrated confidence scores: applying Platt scaling to the LinearSVC outputs would provide better-calibrated probability estimates for the soft-vote ensemble, improving reliability of the confidence scores displayed in the UI.
- Active learning: using the Label Propagation uncertainty estimates to select the most informative unlabeled samples for annotation would reduce labeling cost while improving verifier accuracy.
- Extended evaluation: a human evaluation study using a Likert-scale rubric for distractor plausibility and hint helpfulness would complement the automated BLEU/ROUGE-L/METEOR metrics.
- Add DVC or automate using GitHub Actions

8.3 Ethical Considerations

Building this system required us to address several real-world challenges beyond just model accuracy:

- Our models were trained on the RACE dataset, which is sourced from Chinese English exams. Because of this, the system risks overfitting to a specific regional exam style. If deployed in a different setting, such as the Pakistani national curriculum, the model might struggle to generalize or miss local cultural nuances.
- We designed the Streamlit UI to be fully usable by students with visual impairments. We explicitly fixed WCAG contrast issues in dark mode (manually overriding the hint panel text to #1f2937 with a gold accent) and used standard widget hierarchies to ensure smooth keyboard navigation.
- This tool is designed to assist teachers, not replace them. Because Model B and our heuristic generators can occasionally produce logical errors or irrelevant distractors, these generated quizzes should never be deployed in an actual exam without a teacher reviewing them first.
- Users need to know how their content is being generated. To prevent automation bias, the UI explicitly flags the exact logic used to create each question (e.g., Branch A, B, or C) and includes clear disclaimers that the AI can make mistakes.

9. Conclusion

This project demonstrates that a carefully engineered traditional machine learning pipeline can address the full quiz generation task on the RACE reading comprehension dataset without employing any neural network component. Model A combines OHE cosine similarity sentence extraction, 25 Wh-word templates, and a LinearSVC ranker for question generation, together with a soft-voting ensemble of Logistic Regression, LinearSVC, and Complement Naive Bayes for answer verification. Unsupervised K-Means clustering and semi-supervised Label Propagation provide complementary analyses of the OHE feature space structure. Model B uses TF-IDF cosine similarity with MMR diversification, frequency-based substitution, and optional Word2Vec nearest-neighbor lookup for distractor generation, and a five-feature Logistic Regression scorer for extractive three-tier hint generation. The Streamlit interface unifies all components into a four-screen interactive application with real-time performance monitoring.

The system confirms that handcrafted lexical features, when combined with a well-regularized ensemble of classical classifiers, can achieve meaningful answer verification accuracy on a large-scale multiple-choice benchmark. The interpretability and low inference latency of the traditional ML components make this architecture suitable for deployment in resource-constrained educational settings where neural model serving is not feasible.

References

- [1] Lai, G., Xie, Q., Liu, H., Yang, Y., and Hovy, E. (2017). RACE: Large-scale ReAding Comprehension Dataset From Examinations. In Proceedings of EMNLP 2017.
- [2] Heilman, M., and Smith, N. A. (2010). Good Question! Statistical Ranking for Question Generation. In Proceedings of NAACL-HLT 2010.
- [3] Du, X., Shao, J., and Cardie, C. (2017). Learning to Ask: Neural Question Generation for Reading Comprehension. In Proceedings of ACL 2017.
- [4] Guo, Q., Zhu, X., Chao, W., and Ye, Z. (2016). Generating Distractors for Reading Comprehension Questions from Real Examinations. In Proceedings of AAAI 2016.
- [5] Jiang, Y., and Lee, H.-Y. (2017). What Click-Through Data Can Tell Us About Distractor Quality in Multiple-Choice Questions. In Proceedings of the BEA Workshop 2017.
- [6] Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). BLEU: A Method for Automatic Evaluation of Machine Translation. In Proceedings of ACL 2002.
- [7] Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. In Proceedings of the ACL Text Summarization Workshop 2004.
- [8] Banerjee, S., and Lavie, A. (2005). METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for MT and/or Summarization 2005.
- [9] Zhu, X., and Ghahramani, Z. (2002). Learning from Labeled and Unlabeled Data with Label Propagation. Technical Report CMU-CALD-02-107, Carnegie Mellon University.